IC526 Advanced Compilers
Lec 14:
Compiler Correctness

Yoonseung Kim
DGIST EECS
Fall 2026

Syntax and Semantics

Syntax and Semantics

- A programming language consists of syntax and semantics.

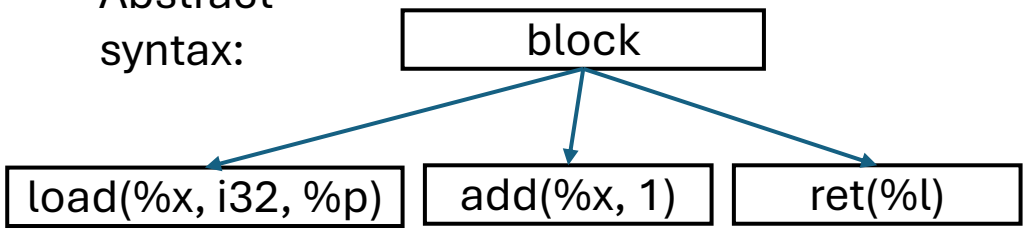
Syntax

Which structure forms a valid program?

Concrete syntax:

```
%x = load i32, ptr %p
%l = add i32 %x, 1
ret i32 %l
```

Abstract syntax:



Semantics

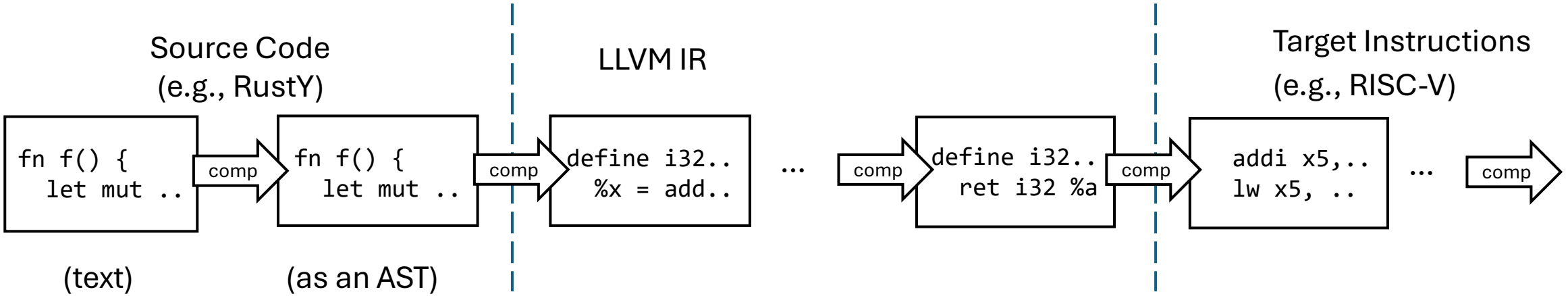
What does a program mean?

Registers: name ↦ (type, value)
Memory: address ↦ byte

rs(%p) = (ptr, a)
read_i32(m, a) = v

$$\frac{\texttt{\%x = load i32, ptr \%p}}{(rs, m) \quad\quad (rs[\%x \mapsto (i32, v)], m)}$$

A (Rough) Understanding of Compilation

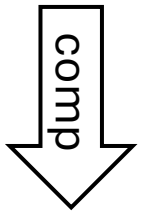


Syntactic structure changes

Observable behavior is preserved

What Is Observable Behavior?

```
fn f(x: i32) -> i32 {
  return g(x + 1);
}
```



Uses a private stack slot

```
define i32 @f(i32 %x) {
  %x.addr = alloca i32
  store i32 %x, ptr %x.addr
  %v = load i32, ptr %x.addr
  %a = add i32 %v, 1
  %r = call i32 @g(i32 %a)
  ret i32 %r
}
```

Keeps x in a register

```
define i32 @f(i32 %x) {
  %a = add i32 %x, 1
  %r = call i32 @g(i32 %a)
  ret i32 %r
}
```

Which observations decide correctness?

A Common Observable Semantics

- Compare observations in a shared domain:
  ➔ Return values and external interactions.

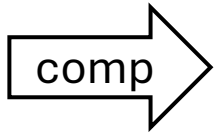
- Each language semantics maps its own **executions** to **observations**.

HW1 already connects two different execution models.

Recall Assignment #1

Source expression

```
Add (Num 1,
     Mul (Num 2, Num 3))
```



Stack-machine program

```
[IPUSH 1; IPUSH 2;
 IPUSH 3; IMUL; IADD]
```

```
compute_expr e = 7
```

```
interp (compile e) = 7
```

```
compute_expr e = interp (compile e)
```

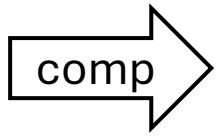
Point: different **executions** expose the same integer result.

What Is Allowed For Optimizations? #1

Both fragments return the same i32 value.

```
...

; return (x + 0)
%a = add i32 %x, 0
ret i32 %a
```




```
...

; return x

ret i32 %x
```

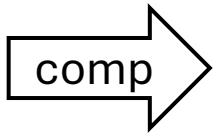
The add takes a step and changes a register.

Which changes are observable, and which are not?

What Is Allowed For Optimizations? #2

A more complex situation:

```
%p = alloca i32
store i32 3, ptr %p
call void @f()
%v = load i32, ptr %p
ret i32 %v
```




```
call void @f()

ret i32 3
```

The local slot changes memory, but its address never escapes.

What if f() does something hacky to access %p's slot?
→ undefined behavior (later)

Observable Behavior

Observable Behavior

The observable behavior of a program is a set of ***traces***:

- A sequence of observable events generated from an execution.

```
fn main() -> i32 {
  println!("{}", foo(2));
  return 0;
}

fn foo(x: i32) -> i32 {
  return x + 1;
}
```

```
Beh(P) = {
  print(3) · return(0)
}
```

Forms of Observable Traces

`Term(t, r)`	`t · return(r)` Finite event sequence, then termination.
`Diverge(t)`	`t · ↑` Finite event sequence, then silent divergence.
`Reactive(T)`	`T = print(1) · print(1) · ...` Infinite event sequence.

One Program, Several Observable Traces

Different environment responses give different traces.

```
fn main() -> i32 {
  let n: i32 = read();
  if n == 0 { return 0; }
  if n == 1 {
    while true { }
  }
  while true {
    println!("{}", 1);
  }
}
```

Possible traces

```
read() ↦ 0 · return(0)

read() ↦ 1 · ↑

read() ↦ 2 · print(1) ·
             print(1) · ...
```

* read() receives an integer from the environment.

Undefined Behavior: When the Program Goes Wrong

```
fn main() -> i32 {
  let scores: [i32; 3] = [70, 80, 90];
  let bonus: i32 = 10;
  let mut total: i32 = 0;
  let mut i: usize = 0;

  while i <= 3 {
    total = total + scores[i];
    i = i + 1;
  }

  println!("{}", total);
  println!("{}", bonus);
  return 0;
}
```

Point: It's intractable to reflect every abnormal situation in the behavior.

One possible stack layout

```
scores:  [70][80][90]
bonus:               [10]
                     ^
                  scores[3]   total: 250
```

Another stack layout

Uninitialized padding

```
scores:  [70][80][90][37]
bonus:                  [10]
                        ^
                     scores[3]   total: 277
```

Or,

```
scores:       [70][80][90]
bonus:   [10]
                        ^
                     scores[3]   segfault
```

Undefined Behavior as All Traces

After an out-of-bounds access, the language makes no guarantees.

```
Beh(P) = All_Traces ∋

        print(250) · print(10) · return(0)
        print(999) · return(0)
        ↑
        print(1) · print(1) · ...
        ...
```

LLVM's Way of Handling Undefined Behavior

LLVM Integer Arithmetic and nsw

i8 uses 8 bits. Signed values range from -128 to 127.

```
add i8 127, 1                                   -128
```

Plain add wraps modulo 256.

```
add nsw i8 127, 1                               poison
```

nsw = No Signed Wrap. Signed overflow produces poison.

Poison and Immediate UB

Poison can remain unused

```
%v = add nsw i8 127, 1
ret i32 0
```

This returns 0.

Poison used by a branch

```
%v = add nsw i8 127, 1
%c = icmp eq i8 %v, 0
br i1 %c, label %yes,
          label %no
```

The branch triggers UB.

Why Poison? Speculative Computation

Assume c = false whenever x = 127.

Before: conditional computation

```
if c:
    return x + 1
else:
    return 0
```

After: compute, then select

```
%v = add nsw i8 %x, 1
%r = select i1 %c,
             i8 %v, i8 0
ret i8 %r
```

The `add` produces poison, but `select` chooses 0 on this input.

Undef and Poison Behave Differently

undef represents an unspecified bit pattern.

```
%a = and i8 undef, 0
```

0

Every possible bit pattern produces zero after masking.

```
%b = and i8 poison, 0
```

poison

Next: Which Behavior Changes Are Allowed?

A source language may permit several observable traces.

```
print(1) · print(2) · return(3)
print(2) · print(1) · return(3)
```

Must compiled code preserve both possibilities?

Next lecture: refinement, simulation, and Alive2.